



DRONE CAMP

DAY 5

Functions, strings, lists, and Boolean logic.

PYTHON + MISSION BRIEFINGS + CONTROL FLOW

MISSION

Turn route text into a mission briefing

The program begins with ordinary text such as forward, left, up, flip. It cleans the text, converts it into a list, separates valid commands from unknown commands, evaluates the route, and prints a complete briefing. Each section adds one tool from the CodingBat Python references.

Input	Transformation	Output
Pilot name	strip(), title(), slice	Clean display name
Comma-separated route	split(), lower(), list append	Command list
Command list	loops + if/elif/else	Difficulty and warnings
Battery text	isdigit() + int()	Ready or revise status

TARGET OUTPUT - WHAT MAIN.PY WILL PRODUCE

```
Pilot name:  aisha
Route:      forward, LEFT, up, flip
Battery:    75
```

```
DRONE CAMP - DAY 5
Pilot: Aisha
Battery: 75%
Difficulty: intermediate
Commands: 4
```

```
ROUTE
1. forward
2. left
3. up
4. flip
```

```
WARNINGS
No warnings found.
```

```
STATUS: READY FOR INSTRUCTOR CHECK
```

File map

- Constants store shared settings and allowed command names.
- Small functions clean strings, validate lists, and return results.
- build_briefing() combines smaller results into one report.
- main() asks for input and controls the order of function calls.

Build rule

Type and test each checkpoint before adding the next function. A small working program is easier to debug than a large untested file.

CHECKPOINT 01

Functions receive values and return results

A function definition names a reusable job. Parameters receive values from the function call. `return` sends a result back and ends that function immediately. When no `return` runs, Python produces the special value `None`.

FUNCTION DEFINITIONS - TYPE THESE BELOW THE CONSTANTS

```
def normalize_name(name):  
    cleaned = name.strip().title()  
  
    if cleaned == "":  
        return "Pilot"  
  
    return cleaned[:20]  
  
def normalize_command(command):  
    return command.strip().lower()
```

Trace one call

Step	Expression	Result
1	<code>normalize_name(" aisha ")</code>	Function begins
2	<code>name.strip()</code>	"aisha"
3	<code>.title()</code>	"Aisha"
4	<code>cleaned[:20]</code>	"Aisha" returned

Why two return statements?

The first `return` handles the empty-name case. If the string is not empty, Python continues to the final `return`. This is often clearer than placing every result inside one large `if/else` block.

Run checkpoint

Temporarily add `print(normalize_name(" sam "))` at the bottom. The output should be `Sam`. Remove the test afterward.

CHECKPOINT 02

Make decisions with Boolean expressions

Comparisons such as `==`, `!=`, `<`, `<=`, `>`, and `>=` produce `True` or `False`. Python joins Boolean expressions with the words `and`, `or`, and `not`. A colon begins each `if`, `elif`, or `else` block, and indentation shows which statements belong to that branch.

IF / ELIF / ELSE + BOOLEAN RETURN

```
def calculate_difficulty(commands):
    points = len(commands)

    for command in commands:
        if command in TURN_COMMANDS:
            points += 1
        elif command in VERTICAL_COMMANDS:
            points += 2
        elif command == "flip":
            points += 3

    if points <= 7:
        return "starter"
    elif points <= 13:
        return "intermediate"
    else:
        return "advanced"

def mission_ready(valid_commands, unknown_commands, battery):
    has_route = len(valid_commands) > 0
    commands_known = len(unknown_commands) == 0
    battery_ready = battery >= MIN_BATTERY

    return has_route and commands_known and battery_ready
```

Read the final return as a sentence

The mission is ready only when it has at least one valid command and no unknown commands and enough battery. If any one condition is `False`, the entire `and` expression becomes `False`.

Boolean note

`0`, an empty string, an empty list, and `None` behave as `False` in an `if`-test. This project uses explicit length comparisons so the condition remains easy to read.

CHECKPOINT 03

Use strings as indexed, immutable text

Strings are written inside single or double quotes. `len(text)` returns the number of characters. Indexes begin at 0, so the final valid index is `len(text) - 1`. A slice `text[start:end]` includes start but stops before end.

STRING INDEXES, SLICES, AND METHODS

```
pilot = "Aisha"
len(pilot)    # 5
pilot[0]      # "A"
pilot[-1]     # "a"
pilot[:2]     # "Ai"
pilot[2:]     # "sha"

cleaned = "  AISHA  ".strip().title()
short_name = cleaned[:20]
```

Strings do not change in place

A string is immutable. Methods such as `lower()`, `title()`, and `strip()` create new strings, so the result must be returned or assigned to a variable. The project stores each cleaned result before using it.

STRING -> LIST - COMMA-SEPARATED ROUTE

```
def split_route(route_text):
    pieces = route_text.split(",")
    commands = []

    for piece in pieces:
        command = normalize_command(piece)

        if command != "":
            commands.append(command)

    return commands
```

Predict first

What list should `split_route(" forward, LEFT, , up ")` return? Write the answer before running the function.

CHECKPOINT 04

Loop through strings when every character matters

A direct loop visits character values one at a time. An index loop visits the positions 0 through len(text) - 1. Use the direct form when only the character matters. Use range(len(text)) when the position or nearby characters matter.

TWO STRING LOOP PATTERNS - SCRATCH TEST

```
message = "flight"

for character in message:
    print(character)

for index in range(len(message)):
    print(index, message[index])
```

Loop form	Best use	Available value
for character in text	Inspect each character	character
for index in range(len(text))	Use positions or neighbors	index and text[index]

Avoid invalid indexes

A direct index outside the string raises an error. Slices are more forgiving: an end value beyond the string simply stops at the real end. This is why cleaned[:20] safely limits a name even when the name is shorter than twenty characters.

STRING CHECK -> INTEGER CONVERSION

```
def read_battery():
    battery_text = input("Battery percentage: ").strip()

    if battery_text.isdigit():
        return int(battery_text)

    print("Battery must be a whole number. Using 0.")
    return 0
```

Run checkpoint	Test 75, 0, and seventy-five. Explain why only the first two strings pass isdigit().
----------------	--

CHECKPOINT 05

Store route commands in lists

A list is an ordered collection. `len(commands)` reports its size, `commands[0]` accesses the first element, and `append(value)` adds an element to the end. Unlike strings, lists are mutable: an existing element can be replaced.

FILE LEVEL - LIST CONSTANTS

```
ALLOWED_COMMANDS = [
    "forward",
    "back",
    "left",
    "right",
    "up",
    "down",
    "cw",
    "ccw",
    "flip",
]

print(len(ALLOWED_COMMANDS))
print(ALLOWED_COMMANDS[0])
print(ALLOWED_COMMANDS[-1])
print(sorted(ALLOWED_COMMANDS))
```

ONE INPUT LIST -> TWO RESULT LISTS

```
def validate_route(commands):
    valid = []
    unknown = []

    for command in commands:
        if command in ALLOWED_COMMANDS:
            valid.append(command)
        else:
            unknown.append(command)

    return valid, unknown
```

Why return two lists?

Python can return more than one value. The call `valid_commands, unknown_commands = validate_route(commands)` stores each returned list in a separate variable.

List note

`sorted(ALLOWED_COMMANDS)` returns a new sorted list. The original command order remains unchanged.

CHECKPOINT 06

Choose direct loops or index loops

LIST VALUES VS LIST INDEXES

```
def count_command(commands, target):  
    count = 0  
  
    for command in commands:  
        if command == target:  
            count += 1  
  
    return count  
  
def build_numbered_route(commands):  
    lines = []  
  
    for index in range(len(commands)):  
        number = index + 1  
        command = commands[index]  
        lines.append(f"{number}. {command}")  
  
    return lines
```

Why range(len(commands)) here?

The numbered route needs both the command and its position. `range(len(commands))` produces every valid index. The displayed number is `index + 1` because people count steps from 1 even though Python indexes begin at 0.

APPEND RESULTS ONLY WHEN A CONDITION IS TRUE

```
def build_warnings(valid_commands, unknown_commands):  
    warnings = []  
  
    if len(valid_commands) == 0:  
        warnings.append("No valid movement commands were entered.")  
  
    if len(unknown_commands) > 0:  
        joined = ", ".join(unknown_commands)  
        warnings.append(f"Unknown commands: {joined}")  
  
    if count_command(valid_commands, "flip") > 1:  
        warnings.append("More than one flip needs an instructor check.")  
  
    if len(valid_commands) > 10:  
        warnings.append("Long route: preview every step before takeoff.")  
  
    return warnings
```

Index safety

The greatest valid list index is `len(commands) - 1`. A loop that uses `commands[index + 1]` must stop one cycle early.

COMPLETE MAIN.PY - PART 1

Constants, cleaning, validation, and route analysis

LINES 1-89 - TYPE IN THIS ORDER

```

TITLE = "DRONE CAMP - DAY 5"
MIN_BATTERY = 40

ALLOWED_COMMANDS = [
    "forward",
    "back",
    "left",
    "right",
    "up",
    "down",
    "cw",
    "ccw",
    "flip",
]

TURN_COMMANDS = ["cw", "ccw"]
VERTICAL_COMMANDS = ["up", "down"]

def normalize_name(name):
    cleaned = name.strip().title()
    if cleaned == "":
        return "Pilot"
    return cleaned[:20]

def normalize_command(command):
    return command.strip().lower()

def split_route(route_text):
    pieces = route_text.split(",")
    commands = []
    for piece in pieces:
        command = normalize_command(piece)
        if command != "":
            commands.append(command)
    return commands

def validate_route(commands):
    valid = []
    unknown = []
    for command in commands:
        if command in ALLOWED_COMMANDS:
            valid.append(command)
        else:
            unknown.append(command)
    return valid, unknown

def count_command(commands, target):
    count = 0
    for command in commands:
        if command == target:
            count += 1
    return count

def calculate_difficulty(commands):
    points = len(commands)
    for command in commands:
        if command in TURN_COMMANDS:
            points += 1
        elif command in VERTICAL_COMMANDS:
            points += 2
        elif command == "flip":
            points += 3
    if points <= 7:
        return "starter"
    elif points <= 13:
        return "intermediate"
    else:
        return "advanced"

def build_numbered_route(commands):
    lines = []
    for index in range(len(commands)):
        number = index + 1
        command = commands[index]
        lines.append(f"{number}. {command}")
    return lines

def build_warnings(valid_commands, unknown_commands):
    warnings = []
    if len(valid_commands) == 0:
        warnings.append("No valid movement commands were entered.")
    if len(unknown_commands) > 0:
        joined = ", ".join(unknown_commands)
        warnings.append(f"Unknown commands: {joined}")
    if count_command(valid_commands, "flip") > 1:
        warnings.append("More than one flip needs an instructor check.")
    if len(valid_commands) > 10:
        warnings.append("Long route: preview every step before takeoff.")
    return warnings

```

Checkpoint

Run the file after line 89 with temporary print calls. Confirm that names are cleaned, routes split correctly, and unknown commands are separated.

COMPLETE MAIN.PY - PART 2

Build the briefing and run the program

LINES 90-162 - BRIEFING, INPUT, OUTPUT, MAIN

```
def mission_ready(valid_commands, unknown_commands, battery):
    has_route = len(valid_commands) > 0
    commands_known = len(unknown_commands) == 0
    battery_ready = battery >= MIN_BATTERY
    return has_route and commands_known and battery_ready

def build_briefing(pilot, valid_commands, unknown_commands, battery):
    difficulty = calculate_difficulty(valid_commands)
    warnings = build_warnings(valid_commands, unknown_commands)
    ready = mission_ready(valid_commands, unknown_commands, battery)
    route_lines = build_numbered_route(valid_commands)
    briefing = []
    briefing.append(TITLE)
    briefing.append(f"Pilot: {pilot}")
    briefing.append(f"Battery: {battery}%")
    briefing.append(f"Difficulty: {difficulty}")
    briefing.append(f"Commands: {len(valid_commands)}")
    briefing.append("")
    briefing.append("ROUTE")
    if len(route_lines) == 0:
        briefing.append("No route available.")
    else:
        for line in route_lines:
            briefing.append(line)
    briefing.append("")
    briefing.append("WARNINGS")
    if len(warnings) == 0:
        briefing.append("No warnings found.")
    else:
        for warning in warnings:
            briefing.append(f"- {warning}")
    briefing.append("")
    if ready:
        briefing.append("STATUS: READY FOR INSTRUCTOR CHECK")
    else:
        briefing.append("STATUS: REVISE THE MISSION")
    return briefing

def read_battery():
    battery_text = input("Battery percentage: ").strip()
    if battery_text.isdigit():
        return int(battery_text)
    print("Battery must be a whole number. Using 0.")
    return 0

def print_briefing(lines):
    print("\n" + "=" * 48)
    for line in lines:
        print(line)
    print("=" * 48)

def main():
    print(TITLE)
    print("Build a route with comma-separated commands.")
    print("Allowed commands:")
    print(", ".join(sorted(ALLOWED_COMMANDS)))
    pilot_input = input("\nPilot name: ")
    route_input = input("Route: ")
    pilot = normalize_name(pilot_input)
    commands = split_route(route_input)
    valid_commands, unknown_commands = validate_route(commands)
    battery = read_battery()
    briefing = build_briefing(
        pilot,
        valid_commands,
        unknown_commands,
        battery,
    )
    print_briefing(briefing)

if __name__ == "__main__":
    main()
```

Final tests

- Valid: Aisha / "forward, left, up, flip" / 75
- Unknown command: Sam / "forward, orbit, right" / 80
- Low battery: Kai / "forward, cw, forward" / 20
- Empty fields: blank name / blank route / non-number battery

Source basis

Concept organization adapted and paraphrased from CodingBat: Python Example Code, Python Strings, Python Lists, and Python If Boolean Logic.